Machine Coding: Resizable Split Panes

- **Problem:** Build resizable split panes: a container split into panes with draggable gutters between them, respecting min / max sizes.
- **Tech stack:** React (Vite, JavaScript), plain CSS
- **Time limit:** 60 minutes
- **Often asked at:** Microsoft, Atlassian

PART A: Thought process

Steps that work for any machine coding task:

1. **Understand the requirements**: functional + non-functional, ask clarifying questions, write down assumptions.
2. **List interactions / use cases**: every action the user can take and what should happen. This is your test checklist.
3. **Design the architecture**: components and their responsibilities. Start with one, split only when needed.
4. **Design the state**: the minimum state, who owns it, what is derived.
5. **Define data structures and contracts**: props, example data, function signatures.
6. **Trace the data flow** of the key interaction end to end.
7. **Find the hard parts** early and decide how to solve them before coding.
8. **Build in layers**: a working version first, then one feature at a time, testing as you go.
9. **Cover states and edge cases**.
10. **Wrap up**: explain trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements, questions, assumptions	5 min
Use cases, architecture, state, contracts	7 min
Static layout (flex panes + gutters)	8 min
Pointer drag + clamping (core)	15 min
Min / max, keyboard, a11y	10 min
Persistence, nesting, polish	8 min
Edge cases + wrap-up	7 min

Tip: think out loud. Say what you are doing and why ("I will compute the delta from the drag start, not the last move, so rounding errors don't add up"). The interviewer grades your reasoning as much as your code.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A container shows 2 or more panes side by side (horizontal) or stacked (vertical).
- A gutter (divider) sits between every two panes. Dragging it resizes the two panes next to it.
- Each pane can have a **min** and **max** size in px; dragging never breaks them.
- Works with mouse, touch and pen.
- Keyboard: gutter is focusable; arrow keys resize, Shift for a bigger step, Home / End jump to min / max.
- Double-click a gutter to reset to the default split.
- Panes can be **nested** (e.g. sidebar | (editor / terminal) | preview, like VS Code).
- Remember the layout across reloads (localStorage).

Non-functional requirements

- **Performance:** dragging must stay smooth (60 fps); pane contents should not re-render on every pointer move.
- **Error handling:** bad or missing localStorage data falls back to defaults; impossible min/max combos don't break the layout.
- **Accessibility:** `role="separator"`, `aria-orientation`, `aria-valuenow`, visible focus, keyboard support.
- **Usability:** clear resize cursor, a hit area bigger than the visible 6px bar, no text selection while dragging.
- **Reusability:** one generic `<SplitPane>` component; panes are just its children.
- **Extensibility:** easy to add collapse, snap points or a controlled mode later.

Clarifying questions

1. Only two panes, or any number? (Assume any number.)
2. Horizontal only, or vertical too, and can they be nested? (Both, nested.)
3. Are min/max sizes in px or %? (px: designers think in px.)
4. Should the layout persist across reloads? (Yes, opt-in with a key.)
5. Do we need keyboard and touch support? (Yes.)
6. Should panes collapse when dragged below min? (No, out of scope; mention as an extension.)

Assumptions

- Sizes are stored in **percent** of the space the panes share, so the layout scales with the window.
- Min / max are given in **px** and converted to percent at drag time.
- Dragging a gutter only changes the **two adjacent panes**; others don't move.
- No library (no react-split-pane, no allotment).

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads (no saved layout)	Panes show `initialSizes` (or equal split)
Page loads (saved layout)	Panes restore the saved sizes
Press on gutter and drag	The two neighbour panes resize, following the pointer
Drag past a pane's min	Pane stops at its min, gutter stops moving
Drag past a pane's max	Pane stops at its max
Pointer leaves the gutter / window mid-drag	Drag keeps working (pointer capture)
Release pointer	Drag ends, cursor and selection back to normal
Hover / focus gutter	Gutter highlights, resize cursor shows
Arrow key on focused gutter	Moves boundary by 2% (10% with Shift)
Home / End on focused gutter	Left/top pane jumps to its min / max
Double-click gutter	Resets to default sizes
Resize the browser window	Panes scale proportionally, CSS still keeps px min
Nested split (vertical inside horizontal)	Each split resizes independently
Touch drag on gutter	Resizes, page does not scroll
Saved data is corrupt	Ignored, defaults used, no crash

3. Design the architecture

Module	Responsibility
`App`	Demo layout: an IDE-style nested split
`SplitPane`	Owns sizes, renders panes + gutters, handles drag / keyboard / persistence
`utils/resize.js`	Pure math: clamp a pair of sizes to min / max (`resizePair`), px → %
`App.css`	Flex layout, gutter styles, drag cursor, hit area

```
App
└── SplitPane (horizontal, storageKey="split:main")
    ├── pane: Explorer
    ├── gutter
    ├── pane: SplitPane (vertical, storageKey="split:editor")
    │         ├── pane: Editor
    │         ├── gutter
    │         └── pane: Terminal
```

```
    ├── gutter
    └── pane: Preview
```

Start simple: the gutter is just a `div` inside `SplitPane`. A separate `Gutter` component would need half a dozen props passed down and adds nothing yet. Split it out only if it grows (collapse buttons, tooltips). The math lives in a pure function so it can be unit tested without a DOM.

---

4. Design the state / data model

State or field	Purpose
`sizes: number[]`	Size of each pane in % (the only real layout state)
`dragging: number`	Index of the gutter being dragged, `-1` when idle (for styles)
`drag` (ref)	Drag session: start pointer pos, start sizes, available px, min/max in %
`containerRef` (ref)	DOM node, measured to convert px ↔ %

```
const [sizes, setSizes] = useState(() => (storageKey ? loadSizes(storageKey, defaults) : defaults));
const [dragging, setDragging] = useState(-1);
const containerRef = useRef(null);
const drag = useRef(null); // { i, start, startSizes, avail, mins, maxs }
```

Senior notes

- **Derived, not stored:** `aria-valuenow`, the `split--dragging` class, `flexGrow` per pane and min/max in % are all computed from `sizes` + props.
- **Ref vs state:** the drag session changes on every move but nothing renders from it, so it is a `ref`. Storing it in state would cause extra renders.
- **Ownership:** each `SplitPane` owns its own `sizes`. Nested splits are independent; the parent never knows about the child's sizes.
- **Why %:** px sizes break when the window resizes; % scales for free. Min/max stay in px because that's what the design says.

---

5. Design the data structures and contracts

Component contract

```
<SplitPane
  direction="horizontal"      // 'horizontal' | 'vertical'
  initialSizes={[20, 55, 25]} // % per pane, default: equal split
  minSizes={[150, 250, 120]}  // px per pane, missing = 0
  maxSizes={[400]}            // px per pane, missing = no max
  storageKey="split:main"     // optional: persist to localStorage
>
  <Explorer />
```

```
  <Editor />
  <Preview />
</SplitPane>
```

Panes are plain children. `Children.toArray` gives each a stable key (`.0` , `.1` ...), so React keeps pane state (scroll position, inputs) while sizes change.

Pure function contract

```
// resizePair(sizes, i, delta, mins, maxs) -> newSizes
resizePair([20, 50, 30], 0, 10, [0, 0, 0], [100, 100, 100]); // [30, 40, 30]
resizePair([20, 50, 30], 0, 100, [10, 15, 10], [100, 100, 100]); // [55, 15, 30] (right min wins)
```

- Input: current sizes (%), gutter index `i` , delta (%), mins and maxs (%).
- Output: a new array, or **the same array** when nothing changes (React then skips the render).

Persisted data (`localStorage["split:main"]`): `[20, 55, 25]` . No API is involved.

---

6. Trace the data flow

Dragging the first gutter 120px to the right:

```
pointerdown on gutter 0
  -> setPointerCapture(pointerId)            all moves now go to the gutter
  -> measure container: avail = 1200 - 2*6 = 1188px
  -> drag.current = { i: 0, start: 241, startSizes: [20,55,25], mins/maxs in % }
  -> setDragging(0)                          gutter turns blue, body cursor = col-resize
       |
pointermove clientX = 361
  -> deltaPx = 361 - 241 = 120  ->  deltaPct = 120 / 1188 * 100 = 10.1%
  -> resizePair([20,55,25], 0, 10.1, mins, maxs) -> [30.1, 44.9, 25]
  -> setSizes(...)                           re-render: pane flexGrow 30.1 / 44.9 / 25
       |
useEffect([sizes])
  -> localStorage.setItem('split:main', '[30.1,44.9,25]')
       |
pointerup / lostpointercapture
  -> drag.current = null, setDragging(-1)   styles back to normal
```

---

7. Important technical challenges

Challenge 1: Clamping two panes against min and max at the same time

Problem: Panes [20, 50, 30], mins [10, 15, 10]. The user drags gutter 0 far to the right. If you only clamp the left pane to its max, the right pane goes below its min (negative even). Clamping each pane separately

makes their sum change and the third pane jumps.

Solution: Only the pair moves and their **sum stays fixed**. So one number, the left size `a`, decides everything. Find the legal range for `a` from all four limits and clamp once:

- `a >= minLeft` and `a <= maxLeft`
- `b = total - a >= minRight` → `a <= total - minRight`
- `b <= maxRight` → `a >= total - maxRight`

```
const total = sizes[i] + sizes[i + 1];
const lo = Math.max(mins[i], total - maxs[i + 1]);
const hi = Math.min(maxs[i], total - mins[i + 1]);
if (lo > hi) return sizes; // constraints can't both hold: don't move
const a = Math.min(hi, Math.max(lo, sizes[i] + delta));
```

Challenge 2: Losing the drag when the pointer leaves the gutter

Problem: The gutter is 6px wide. A fast mouse move jumps off it, `mousemove` on the gutter stops firing, and the drag freezes. Releasing the mouse outside the window never fires `mouseup`, so the drag gets stuck on. An iframe or code editor inside a pane swallows the events. On touch, the page scrolls instead of resizing.

Solution: Pointer Events with **pointer capture**: after `setPointerCapture`, every move and the release go to the gutter, wherever the pointer is. `lostpointercapture` is a safe "drag ended" for every case (up, cancel, alt-tab). CSS does the rest:

```
function onPointerDown(e, i) {
  if (e.button !== 0) return;
  e.currentTarget.setPointerCapture(e.pointerId);
  drag.current = { i, start: horizontal ? e.clientX : e.clientY, startSizes: sizes, ...limits() };
  setDragging(i);
}
```

```
.gutter { touch-action: none; }                /* touch resizes, doesn't scroll */
.split--dragging { user-select: none; }        /* no text selection mid-drag */
.split--dragging > .pane { pointer-events: none; } /* iframes can't steal events */
```

The old way (`mousemove` / `mouseup` on `window`, added on mousedown, removed on mouseup) also works, but needs cleanup on unmount and a separate touch path.

Challenge 3: Drift and px ↔ % conversion

Problem: If each move adds the delta since the **last** move, small rounding and clamping errors pile up: drag right into the max, keep going, come back, and the gutter is no longer under the pointer. Also, `%` must be of the space the panes share, not the whole container, or gutters make the math wrong.

Solution: Save `startSizes` and the start pointer position on pointerdown. On each move, compute the delta **from the start** and apply it to `startSizes`. The gutter always sits exactly where the pointer is (or at the

limit). Measure once at drag start: `avail = containerSize - gutters * 6px`, and lay panes out with `flex-grow: size; flex-basis: 0` so flexbox shares exactly that space by the same ratio.

```
const deltaPx = (horizontal ? e.clientX : e.clientY) - d.start;
setSizes(resizePair(d.startSizes, d.i, pxToPct(deltaPx, d.avail), d.mins, d.maxs));
```

```
.pane { flex-basis: 0; min-width: 0; min-height: 0; overflow: auto; }
```

`min-width: 0` matters: a flex item defaults to `min-width: auto` and won't shrink below its content (a long line of code), so the pane would refuse to get smaller.

Challenge 4: Smooth drag without re-rendering pane contents

Problem: `setSizes` runs on every pointer move (60+ times a second). If the editor or a big table inside a pane re-renders each time, the drag stutters.

Solution: Pane contents are `children` created by the parent, which does not re-render during the drag. Their element objects keep the same reference, so React skips them; only the wrapper `div` style changes. `resizePair` returns the **same array** when clamped, so drags past the limit cause no render at all. If it ever still lags: throttle with `requestAnimationFrame`, or write `flex-grow` straight to the DOM during the drag and commit to state on pointerup.

---

8. Implement in the correct order

#	Step	Working result
1	Vite app, `App` with a full-height container	Blank page fills the window
2	`SplitPane` rendering children with gutters between them	Static 3 columns + dividers
3	`sizes` state + `flexGrow` per pane, `min-width: 0`	Sizes driven by state
4	Pointer down / move / up with pointer capture, delta from start	Dragging works (no limits yet)
5	`resizePair` pure function + node asserts	Clamped math tested
6	px min/max → % at drag start, wire into move	Panes stop at min / max
7	Drag styles: cursor, no select, `touch-action`, hit area	Feels native
8	Keyboard + ARIA on the gutter	Accessible
9	`direction="vertical"` + nested demo	IDE layout works
10	Double-click reset + localStorage persistence	Layout remembered
11	CSS min/max on panes for window resize	Shrinking window keeps px limits
12	Walk the edge-case table	Done

---

9. Cover all states and edge cases

State / edge case	Handling
Initial, no saved layout	`initialSizes`, or an equal split if not given
Initial, saved layout	Restored in the lazy `useState` initializer (no flash)
Idle	Gray gutter
Hover / focus	Blue gutter, `col-resize` / `row-resize` cursor
Dragging	`split--dragging`: cursor locked, no text select, panes ignore pointer
Pointer released outside window	Pointer capture still delivers `pointerup` / `lostpointercapture`
Right-click / middle-click on gutter	Ignored (`e.button !== 0`)
Drag past min / max	Clamped; same array returned, no re-render
min + min wider than the pair	`lo > hi`: pane pair doesn't move
Corrupt / wrong-length localStorage	Validated, falls back to defaults
localStorage blocked (private mode)	`try/catch`, layout works, just not saved
Window shrinks	% scale; CSS `min-width` keeps px mins
Container hidden (0px)	`avail` floored at 1 to avoid divide by zero
Long content in a pane	`min-width: 0` + `overflow: auto`: pane scrolls
Iframe / editor inside pane	`pointer-events: none` during drag
Touch device	`touch-action: none` + pointer events
Single child	No gutters rendered; one full pane
Nested splits	Each one owns its own state and storage key

---

10. Final structure

```
_87_resizable_split_panes/
├── index.html
├── package.json
├── vite.config.js
├── check.mjs                # node assert self-check for resize.js
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx              # IDE-style nested demo
    ├── App.css              # split / pane / gutter styles
    ├── components/
    │   └── SplitPane.jsx
```

```
    └── utils/
        └── resize.js         # resizePair, pxToPct
```

11. Full working code

`src/utils/resize.js`

```
export const pxToPct = (px, totalPx) => (px / totalPx) * 100;

// Move the boundary between pane i and i+1 by delta (%).
// Only the two neighbours change, their sum stays the same, and both stay inside their min/max.
export function resizePair(sizes, i, delta, mins, maxs) {
  const total = sizes[i] + sizes[i + 1];
  const lo = Math.max(mins[i], total - maxs[i + 1]);
  const hi = Math.min(maxs[i], total - mins[i + 1]);
  if (lo > hi) return sizes; // constraints can't both hold: don't move
  const a = Math.min(hi, Math.max(lo, sizes[i] + delta));
  if (a === sizes[i]) return sizes; // same reference → React skips the re-render
  const next = [...sizes];
  next[i] = a;
  next[i + 1] = total - a;
  return next;
}
```

`src/components/SplitPane.jsx`

```
import { Children, Fragment, useEffect, useRef, useState } from 'react';
import { pxToPct, resizePair } from '../utils/resize';

const GUTTER = 6; // px, must match .gutter in App.css
const KEY_STEP = 2; // % per arrow key, x5 with Shift

function loadSizes(storageKey, fallback) {
  try {
    const saved = JSON.parse(localStorage.getItem(storageKey));
    if (Array.isArray(saved) && saved.length === fallback.length && saved.every(Number.isFinite)) return
saved;
  } catch {
    // private mode or bad JSON → defaults
  }
  return fallback;
}

export default function SplitPane({
  direction = 'horizontal',
```

```
  initialSizes,
  minSizes = [],
  maxSizes = [],
  storageKey,
  children,
}) {
  const panes = Children.toArray(children);
  const defaults = initialSizes ?? panes.map(() => 100 / panes.length);
  const [sizes, setSizes] = useState(() => (storageKey ? loadSizes(storageKey, defaults) : defaults));
  const [dragging, setDragging] = useState(-1);
  const containerRef = useRef(null);
  const drag = useRef(null); // { i, start, startSizes, avail, mins, maxs } while dragging
  const horizontal = direction === 'horizontal';

  useEffect(() => {
    if (!storageKey) return;
    try {
      localStorage.setItem(storageKey, JSON.stringify(sizes));
    } catch {
      // quota / private mode: layout still works, just not remembered
    }
  }, [storageKey, sizes]);

  // px limits → % of the space the panes share (container minus gutters)
  function limits() {
    const rect = containerRef.current.getBoundingClientRect();
    const avail = Math.max(1, (horizontal ? rect.width : rect.height) - GUTTER * (panes.length - 1));
    return {
      avail,
      mins: panes.map((_, i) => pxToPct(minSizes[i] ?? 0, avail)),
      maxs: panes.map((_, i) => (maxSizes[i] == null ? 100 : pxToPct(maxSizes[i], avail))),
    };
  }

  function onPointerDown(e, i) {
    if (e.button !== 0) return;
    e.currentTarget.setPointerCapture(e.pointerId); // keep getting moves outside the gutter / window
    drag.current = { i, start: horizontal ? e.clientX : e.clientY, startSizes: sizes, ...limits() };
    setDragging(i);
  }

  function onPointerMove(e) {
    const d = drag.current;
    if (!d) return;
    // delta from the drag start, not the last move → no rounding drift
    const deltaPx = (horizontal ? e.clientX : e.clientY) - d.start;
    setSizes(resizePair(d.startSizes, d.i, pxToPct(deltaPx, d.avail), d.mins, d.maxs));
```

```
  }

  function endDrag() {
    drag.current = null;
    setDragging(-1);
  }

  function onKeyDown(e, i) {
    const step = e.shiftKey ? KEY_STEP * 5 : KEY_STEP;
    const deltas = {
      [horizontal ? 'ArrowLeft' : 'ArrowUp']: -step,
      [horizontal ? 'ArrowRight' : 'ArrowDown']: step,
      Home: -100, // clamped to the min
      End: 100, // clamped to the max
    };
    if (!(e.key in deltas)) return;
    e.preventDefault();
    const { mins, maxs } = limits();
    setSizes((s) => resizePair(s, i, deltas[e.key], mins, maxs));
  }

  return (
    <div
      ref={containerRef}
      className={`split split--${direction}${dragging >= 0 ? ' split--dragging' : ''}`}
    >
      {panes.map((pane, i) => (
        <Fragment key={pane.key}>
          {i > 0 && (
            <div
              role="separator"
              tabIndex={0}
              aria-label="Resize panes"
              aria-orientation={horizontal ? 'vertical' : 'horizontal'}
              aria-valuenow={Math.round(sizes[i - 1])}
              aria-valuemin={0}
              aria-valuemax={100}
              className={`gutter${dragging === i - 1 ? ' gutter--active' : ''}`}
              onPointerDown={(e) => onPointerDown(e, i - 1)}
              onPointerMove={onPointerMove}
              onPointerUp={endDrag}
              onPointerCancel={endDrag}
              onLostPointerCapture={endDrag}
              onKeyDown={(e) => onKeyDown(e, i - 1)}
              onDoubleClick={() => setSizes(defaults)}
            />
          )}
```

```
          <div
            className="pane"
            style={{
              flexGrow: sizes[i],
              // CSS enforces px limits too, so a shrinking window can't squash a pane
              [horizontal ? 'minWidth' : 'minHeight']: minSizes[i] ?? 0,
              [horizontal ? 'maxWidth' : 'maxHeight']: maxSizes[i],
            }}
          >
            {pane}
          </div>
        </Fragment>
      ))}
    </div>
  );
}
```

`src/App.jsx`

```
import SplitPane from './components/SplitPane';
import './App.css';

export default function App() {
  return (
    <div className="app">
      <header className="toolbar">
        Resizable split panes: drag a gutter, focus it and use arrow keys (Shift = bigger step, Home/End = min/max), double-click to reset
      </header>
      <main className="workspace">
        <SplitPane
          storageKey="split:main"
          initialSizes={[20, 55, 25]}
          minSizes={[150, 250, 120]}
          maxSizes={[400]}
        >
          <aside className="panel">
            <h2>Explorer</h2>
            <p>min 150px, max 400px</p>
          </aside>
          <SplitPane direction="vertical" storageKey="split:editor" initialSizes={[70, 30]} minSizes={[120, 60]}>
            <section className="panel">
              <h2>Editor</h2>
              <p>min 250px wide, 120px tall</p>
            </section>
            <section className="panel panel--dark">
```

```
              <h2>Terminal</h2>
              <p>min 60px tall</p>
            </section>
          </SplitPane>
          <aside className="panel">
            <h2>Preview</h2>
            <p>min 120px</p>
          </aside>
        </SplitPane>
      </main>
    </div>
  );
}
```

`src/App.css`

```
.app {
  height: 100vh;
  display: flex;
  flex-direction: column;
}

.toolbar {
  padding: 8px 12px;
  font-size: 13px;
  background: #18181b;
  color: #e4e4e7;
}

.workspace {
  flex: 1;
  min-height: 0;
}

/* ---- SplitPane ---- */
.split {
  display: flex;
  width: 100%;
  height: 100%;
  overflow: hidden;
}

.split--vertical {
  flex-direction: column;
}

.pane {
```

```
  flex-basis: 0; /* flex-grow = size %, so panes share the space left after gutters */
  flex-shrink: 1;
  min-width: 0; /* default min-width:auto would stop panes shrinking below their content */
  min-height: 0;
  overflow: auto;
}

.gutter {
  flex: 0 0 6px;
  position: relative;
  background: #d4d4d8;
  touch-action: none; /* touch drag resizes instead of scrolling */
}

/* bigger invisible hit area than the 6px bar */
.gutter::before {
  content: '';
  position: absolute;
  inset: 0 -4px;
  z-index: 1;
}

.split--vertical > .gutter::before {
  inset: -4px 0;
}

.split--horizontal > .gutter {
  cursor: col-resize;
}

.split--vertical > .gutter {
  cursor: row-resize;
}

.gutter:hover,
.gutter:focus-visible,
.gutter--active {
  background: #3b82f6;
  outline: none;
}

.split--dragging {
  user-select: none;
}

.split--horizontal.split--dragging {
  cursor: col-resize;
```

```
}

.split--vertical.split--dragging {
  cursor: row-resize;
}

/* iframes/editors inside panes would swallow pointer events mid-drag */
.split--dragging > .pane {
  pointer-events: none;
}

/* ---- demo panels ---- */
.panel {
  height: 100%;
  padding: 12px;
  background: #fafafa;
}

.panel--dark {
  background: #27272a;
  color: #e4e4e7;
}

.panel h2 {
  margin: 0 0 4px;
  font-size: 15px;
}

.panel p {
  margin: 0;
  font-size: 13px;
  opacity: 0.7;
}
```

`src/index.css`

```
*,
*::before,
*::after {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

`vite.config.js`

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react-swc'

export default defineConfig({
  plugins: [react()],
})
```

`check.mjs` **(run with** `node check.mjs` **)**

```
import assert from 'node:assert/strict';
import { pxToPct, resizePair } from './src/utils/resize.js';

const no = [0, 0, 0];
const all = [100, 100, 100];

assert.deepEqual(resizePair([20, 50, 30], 0, 10, no, all), [30, 40, 30]); // only the pair moves
assert.deepEqual(resizePair([20, 50, 30], 1, -20, no, all), [20, 30, 50]);
assert.deepEqual(resizePair([20, 50, 30], 0, 100, [10, 15, 10], all), [55, 15, 30]); // right min wins
assert.deepEqual(resizePair([20, 50, 30], 0, -100, [10, 15, 10], all), [10, 60, 30]); // left min wins
assert.deepEqual(resizePair([20, 50, 30], 0, 30, no, [25, 100, 100]), [25, 45, 30]); // left max wins
assert.deepEqual(resizePair([20, 50, 30], 0, -15, no, [100, 55, 100]), [15, 55, 30]); // right max wins
const same = [20, 50, 30];
assert.equal(resizePair(same, 0, -5, [20, 0, 0], all), same); // already at min → same ref
assert.equal(resizePair(same, 0, 5, [60, 60, 0], all), same); // impossible mins → no move
assert.equal(pxToPct(150, 600), 25);
console.log('resize checks passed');
```

---

12. Wrap-up

Edge cases handled

- Min and max on both sides of every gutter, clamped together in one step.
- Drag outside the gutter or window, touch, pen, iframes inside panes.
- No drift: delta measured from the drag start.
- Keyboard and screen reader support with `role="separator"`.
- Corrupt or blocked localStorage, window resizing, hidden container, nested splits.

Trade-offs to mention

- **% state vs px state:** % scales with the window but needs a px ↔ % conversion for min/max; px is simpler but breaks on resize.
- **State update per move vs direct DOM writes:** state is simple and React-y; direct `style` writes plus a commit on pointerup are faster for very heavy panes.
- **Persist on every change:** `useEffect` writes localStorage on each move. Small and synchronous, so fine here; with a slow store, write on pointerup only.
- **Uncontrolled component:** easy to drop in, but a parent can't set sizes. A controlled mode (`sizes` + `onChange`) is the next step.
- **CSS min/max vs state:** on window shrink, CSS keeps px limits but `sizes` doesn't change, so state and screen can briefly disagree until the next drag.

With more time

- Collapse a pane on drag below a threshold or with a button, and snap points.
- Controlled mode, `onResizeEnd` callback.
- `ResizeObserver` to re-clamp `sizes` when the container shrinks.
- Unit tests with Vitest + Testing Library (keyboard, double-click), Playwright for real drags.
- `requestAnimationFrame` throttling for heavy pane contents.
- RTL support (flip the delta when `dir="rtl"`).

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why store sizes in percent and not pixels?

Percent scales automatically when the window or a parent pane resizes; px would need recalculating on every container resize. Min/max stay in px because designs give them in px, so we convert them to % using the container size at drag start.

Q2. What if the interviewer wants only two panes? Is N panes over-engineering?

The N-pane version costs almost nothing extra: a `map` over children and an index on the gutter. The pair-wise `resizePair` is the same math either way. If time is tight, hardcode two panes first and generalize in the last 5 minutes.

Q3. What if they ask for collapsible panes?

Add a `collapsible` flag per pane. On drag below `min / 2`, snap the size to 0 and remember the previous size; dragging out or double-clicking restores it. The clamp range just gains one more allowed value (0) besides `[lo, hi]`.

State design

Q4. Why is the drag session a ref and not state?

Nothing renders from it. Putting it in state would schedule a render for data the UI never shows, and a stale closure could read an old value. The ref is read and written synchronously inside event handlers.

Q5. Why not let the parent own the sizes (controlled component)?

For the demo, uncontrolled is simpler to use. In a real library I'd support both like `<input>`: `sizes` + `onChange` for controlled, `defaultSizes` for uncontrolled. The internal code barely changes: replace `setSizes` with a function that calls `onChange` when controlled.

Q6. Why use a lazy `useState` initializer for localStorage?

`useState(() => loadSizes(...))` runs only on the first render, so we don't parse JSON on every drag move, and the saved layout shows on the very first paint. Reading it in a `useEffect` would flash the default layout first.

Hard part: clamping

Q7. Why clamp the pair together instead of each pane separately?

The pair's sum must stay constant, or the third pane moves. Clamping the left pane alone ignores the right pane's min. With the sum fixed, one variable decides both sizes, so all four limits become one range `[lo, hi]` for that variable.

Q8. What if the min sizes of all panes add up to more than the container?

`lo > hi`, so `resizePair` refuses to move. Visually, CSS `min-width` makes the panes overflow and the container clips (`overflow: hidden`). Better options: let the container scroll, or collapse the lowest-priority pane. I'd ask the interviewer which one the product wants.

Q9. Dragging gutter 0 hard against a min: should it push gutter 1 along?

Some tools (VS Code) do "push" resizing: when a pane hits min, the next pane starts shrinking. It's a loop: apply what's left of the delta to the next pair. I kept pair-only for predictability; it's a good extension.

Hard part: pointer handling

Q10. Why pointer events with `setPointerCapture` instead of mouse events on `window`?

One code path for mouse, touch and pen; capture keeps events coming even outside the window; `lostpointercapture` covers every way a drag can end; no global listeners to clean up on unmount. With mouse events you'd add/remove `window` listeners and write a separate touch handler.

Q11. The panes contain an iframe and the drag freezes over it. Why, and the fix?

The iframe is a separate document, so pointer events over it go to the iframe, not our page. Pointer capture usually fixes it, but as a safety net we set `pointer-events: none` on panes while dragging, which is what `.split--dragging > .pane` does.

Q12. Why does the gutter need `touch-action: none`?

Without it, a finger drag on touch screens starts a page scroll / pan, and the browser fires `pointercancel`, ending our drag. `touch-action: none` tells the browser we handle the gesture.

Implementation steps

Q13. Why `flex-grow` with `flex-basis: 0` instead of `width: X%`?

Flexbox shares the space **after** the gutters by the `flex-grow` ratio, so sizes always fit exactly: no `calc(20% - 4px)` math per pane. It also respects `min-width` / `max-width` and redistributes the rest automatically when the window shrinks.

Q14. Why is `min-width: 0` on panes needed?

Flex items default to `min-width: auto`, which means "not smaller than my content". A long unbroken line inside a pane would then block it from shrinking and push other panes. `min-width: 0` plus `overflow: auto` lets the pane shrink and scroll.

Q15. How did you make the 6px gutter easy to grab?

A `::before` pseudo-element with `inset: 0 -4px` and `z-index: 1` makes a 14px invisible hit area. The bar looks thin but is easy to hit, including with a finger.

Q16. How does the gutter work for screen readers and keyboard users?

It's `role="separator"` with `tabIndex=0`, `aria-orientation` (vertical bar for a horizontal split), `aria-valuenow` with the left pane's %, and `aria-label`. Arrow keys move it, Shift for big steps, Home/End for min/max, matching the WAI-ARIA window splitter pattern.

Performance

Q17. What if each pane holds a heavy component (a 10k-row grid)?

Children are elements made by the parent, which doesn't re-render during the drag, so React skips them. If the browser's layout itself is slow, write `flex-grow` directly to the DOM in `pointermove` (inside `requestAnimationFrame`) and commit to React state on `pointerup`.

Q18. What if the pointer fires 240 times a second on a high-refresh mouse?

React batches, but each move still sets state. Throttle with `requestAnimationFrame`: store the latest event in a ref and process it once per frame. Also, clamped moves return the same array, so moves past the limit don't render at all.

Q19. Writing localStorage on every pointer move: is that a problem?

`setItem` is synchronous but the payload is a few bytes, so it's cheap here. With many panes or a slow store (IndexedDB, server), save only on `pointerup` / keydown, or debounce the write.

Trade-offs and "why not X?"

Q20. Why not use the CSS `resize` property or a library like react-split-pane / allotment?

CSS `resize` only drags a corner, has no gutter, no min on the neighbour and no keyboard support. A library is the right call in production, but the interview is testing whether you can build the drag, clamping and a11y yourself.

Q21. What if the container itself resizes (sidebar toggle) while not dragging?

% sizes scale with it, and CSS `min-width` / `max-width` keep the px limits on screen. To keep state and screen in sync, add a `ResizeObserver` on the container and re-clamp `sizes` with the new px → % limits.

Q22. How would you test this?

Unit-test `resizePair` (done with node asserts: pair-only, each of the four limits, impossible limits, same reference). Component tests with Testing Library for keyboard, double-click and persistence. Real drag with Playwright `mouse.down/move/up`, because jsdom has no layout (`getBoundingClientRect` returns zeros).

Q23. How would you support right-to-left layouts?

In RTL, a horizontal flex row is mirrored, so moving the pointer right should shrink the first pane. Read `getComputedStyle(container).direction` at drag start and negate the delta (and swap ArrowLeft / ArrowRight) when it's `rtl`.